

Real-Time Traffic Anomaly Detection

Enriched with Weather Data via Apache Flink Stream Join + Gemini Insight Generation

Dokumen ini merangkum use case hackathon: pipeline streaming yang menggabungkan dua sumber data (trip/traffic events dan weather events) menggunakan Apache Flink, lalu menghasilkan insight berbahasa natural menggunakan Gemini LLM. Use case ini dirancang untuk menyasar dua kategori penilaian: **Most Flink-Driven App** dan **Most Impactful AI App**.

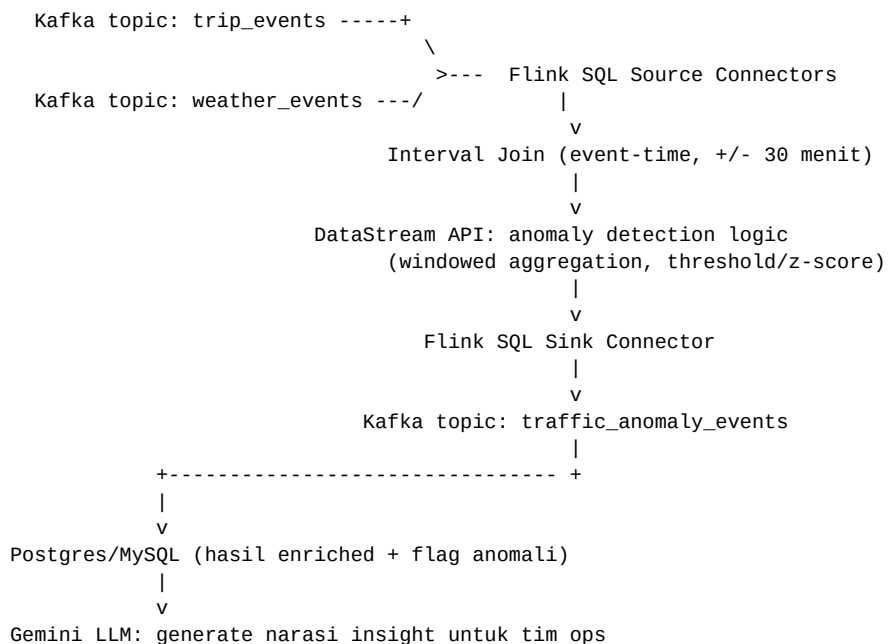
1. Ringkasan Use Case

Ide dasar: replay data historis trip (NYC TLC) dan data cuaca sebagai dua stream terpisah, lalu Flink melakukan **interval join** berbasis event-time antara keduanya. Hasil join dipakai untuk mendeteksi anomali (misal penurunan volume trip atau kenaikan fare drastis saat cuaca ekstrem), dan Gemini menerjemahkan hasil deteksi menjadi narasi yang mudah dipahami tim non-teknis.

Value proposition-nya: menunjukkan Flink benar-benar dipakai untuk stateful stream processing (join dua stream + windowed aggregation), bukan sekadar passthrough data dari Kafka ke database.

2. Arsitektur Pipeline

```
Postgres (trip_events) --CDC (Debezium)--> Kafka topic: trip_events (Avro + Schema Registry)
Python producer (weather) --produce langsung-> Kafka topic: weather_events (Plain JSON, no registry)
```



Catatan: hop terakhir langsung dari Flink SQL Sink Connector ke Postgres/MySQL (JDBC), tanpa lewat topic tambahan lagi, kecuali dibutuhkan fan-out ke lebih dari satu consumer.

3. Sumber Data

3.1 Trip Events (Topic 1)

Sumber: NYC TLC Trip Record Data (parquet, publik). Direplay ke Postgres menggunakan script simulator Python yang men-scale gap antar timestamp asli dengan faktor kecepatan tertentu, sehingga insert ke Postgres terasa seperti stream realtime. Dari Postgres, CDC (Debezium) menangkap perubahan dan mempublikasikannya ke topic Kafka dalam format Avro dengan skema terdaftar di Schema Registry.

3.2 Weather Events (Topic 2)

Sumber: data cuaca historis (misal Open-Meteo atau NOAA) per area/waktu, dipublikasikan langsung ke Kafka oleh producer Python sederhana (bukan lewat Postgres/CDC sama sekali). Format pesan-nya **plain JSON**, tanpa Schema Registry. Ini sengaja dibuat berbeda dari topic pertama untuk menunjukkan Flink bisa menyatukan sumber dengan producer dan format serialisasi yang sama sekali berbeda dalam satu job.

Konsekuensinya: tidak ada schema evolution safety net seperti Avro+registry — field JSON harus match persis dengan DDL Flink (nama, tipe, dan format timestamp ISO-8601), kalau tidak job akan error saat runtime, bukan ter-deteksi lebih awal.

4. Flink SQL: Source Table Definitions

Table **trip_events** berasal dari CDC Debezium, sehingga menggunakan format `debezium-avro-confluent` yang otomatis unwrap envelope Debezium (`before/after/op/source`) menjadi changelog stream native Flink.

```
CREATE TABLE trip_events (
  id                BIGINT,
  pickup_datetime   TIMESTAMP(3),
  pu_location_id    INT,
  trip_distance     DOUBLE,
  fare_amount       DOUBLE,
  WATERMARK FOR pickup_datetime AS pickup_datetime - INTERVAL '5' MINUTE,
  PRIMARY KEY (id) NOT ENFORCED
) WITH (
  'connector' = 'kafka',
  'topic' = 'trip_events',
  'properties.bootstrap.servers' = 'kafka:9092',
  'format' = 'debezium-avro-confluent',
  'debezium-avro-confluent.schema-registry.url' = 'http://schema-registry:8081'
);
```

Table **weather_events** diproduksi langsung oleh producer Python (bukan CDC, bukan Avro), jadi cukup format `json` — tidak butuh Schema Registry sama sekali.

```
CREATE TABLE weather_events (
  location_id      INT,
  event_time       TIMESTAMP(3),
  condition        STRING,
  precipitation     DOUBLE,
  WATERMARK FOR event_time AS event_time - INTERVAL '5' MINUTE
) WITH (
  'connector' = 'kafka',
  'topic' = 'weather_events',
  'properties.bootstrap.servers' = 'kafka:9092',
  'format' = 'json',
  'json.timestamp-format.standard' = 'ISO-8601'
);
```

Field JSON dari producer Python harus persis: `event_time` dikirim sebagai string ISO-8601 (misal `2024-01-15T17:00:00`), bukan epoch millis, kecuali `json.timestamp-format.standard` diset ke SQL dan formatnya disesuaikan.

5. Interval Join: Enrich Trip dengan Cuaca

Interval join mensyaratkan watermark pada kedua stream (sudah didefinisikan di DDL di atas), dan kondisi join harus membatasi jarak waktu antar event agar Flink bisa membuang state yang sudah tidak relevan (bukan join tanpa batas waktu).

```
SELECT
  t.pickup_datetime,
  t.pu_location_id,
  t.trip_distance,
  t.fare_amount,
  w.condition,
  w.precipitation
```

```

FROM trip_events t
JOIN weather_events w
  ON t.pu_location_id = w.location_id
  AND t.pickup_datetime BETWEEN w.event_time - INTERVAL '30' MINUTE
                                AND w.event_time + INTERVAL '30' MINUTE;

```

6. Menggabungkan Table API dan DataStream API (PyFlink)

Hasil interval join di atas berbentuk Table (SQL). Untuk masuk ke custom logic anomaly detection di DataStream API, perlu konversi dua arah dalam satu StreamTableEnvironment: Table → DataStream untuk masuk ke custom logic, lalu DataStream → Table lagi untuk kembali ke SQL Sink Connector.

6.1 Table -> DataStream

Karena hasil interval join adalah stream append-only (bukan hasil aggregation yang bisa update/delete), cukup pakai `to_data_stream()`. Kalau sumbernya changelog (ada update/delete, misal langsung dari `trip_events` yang masih mentah), harus pakai `to_changelog_stream()` supaya flag INSERT/UPDATE/DELETE ikut terbawa, bukan cuma `to_data_stream()` yang mengasumsikan append-only.

```

from pyflink.table import StreamTableEnvironment
from pyflink.datastream import StreamExecutionEnvironment

env = StreamExecutionEnvironment.get_execution_environment()
t_env = StreamTableEnvironment.create(env)

# ... DDL trip_events & weather_events, lalu interval join ...
joined_table = t_env.sql_query("""
    SELECT t.pickup_datetime, t.pu_location_id, t.trip_distance,
           t.fare_amount, w.condition, w.precipitation
    FROM trip_events t
    JOIN weather_events w
      ON t.pu_location_id = w.location_id
      AND t.pickup_datetime BETWEEN w.event_time - INTERVAL '30' MINUTE
                                AND w.event_time + INTERVAL '30' MINUTE
""")

# Table -> DataStream (append-only, karena interval join tidak menghasilkan retract/update)
ds = t_env.to_data_stream(joined_table)

```

6.2 Custom logic di DataStream API

Di level DataStream, row dari Table API muncul sebagai objek `Row` (`pyflink.common.Row`), diakses lewat index atau nama field. Windowed aggregation/anomaly logic ditulis pakai `key_by + window + process()` seperti biasa:

```

from pyflink.common import Row, Time
from pyflink.datastream.window import TumblingEventTimeWindows
from pyflink.datastream.functions import ProcessWindowFunction

class AnomalyDetector(ProcessWindowFunction):
    def process(self, key, context, elements, out):
        rows = list(elements)
        avg_fare = sum(r.fare_amount for r in rows) / len(rows)
        trip_count = len(rows)
        # logic threshold/z-score dibandingkan baseline historis (disimpan di state)
        if trip_count < BASELINE_MIN_COUNT: # contoh sederhana
            out.collect(Row(
                location_id=key, window_end=context.window().end,
                trip_count=trip_count, avg_fare=avg_fare, is_anomaly=True,
            ))

anomaly_stream = (
    ds.key_by(lambda r: r.pu_location_id)
      .window(TumblingEventTimeWindows.of(Time.minutes(15)))
      .process(AnomalyDetector())
)

```

6.3 DataStream -> Table (kembali ke SQL Sink)

Hasil `anomaly_stream` dikonversi balik jadi Table dengan `from_data_stream()`, lalu di-INSERT INTO ke table sink yang sudah didefinisikan lewat SQL Sink Connector.

```
anomaly_table = t_env.from_data_stream(
    anomaly_stream,
    Schema.new_builder()
        .column("location_id", DataTypes.INT())
        .column("window_end", DataTypes.TIMESTAMP(3))
        .column("trip_count", DataTypes.BIGINT())
        .column("avg_fare", DataTypes.DOUBLE())
        .column("is_anomaly", DataTypes.BOOLEAN())
        .build()
)

t_env.create_temporary_view("anomaly_result", anomaly_table)
t_env.execute_sql("""
    INSERT INTO traffic_anomaly_events
    SELECT location_id, window_end, trip_count, avg_fare, is_anomaly
    FROM anomaly_result
""")
```

Catatan penting: watermark yang sudah dibangun di level Table API (dari DDL `WATERMARK FOR ...`) otomatis terbawa saat konversi ke DataStream, jadi windowed processing di section 6.2 tetap event-time correct tanpa perlu define watermark strategy ulang secara manual.

7. DataStream API: Deteksi Anomali

Hasil interval join dialirkan ke DataStream API untuk logic yang tidak praktis dilakukan murni via SQL, misalnya:

- Windowed aggregation per area (rolling count/avg fare per 15 menit) untuk baseline normal.
- Deteksi deviasi (misal z-score) dari baseline saat kondisi cuaca tertentu (hujan deras, badai).
- Emit event ke topic `traffic_anomaly_events` hanya saat deviasi melewati threshold, sehingga downstream tidak dibanjiri event redundan.

Bagian ini yang menunjukkan Flink dipakai secara stateful (bukan cuma SQL passthrough) — cocok untuk kategori **Most Flink-Driven App**, karena mengombinasikan SQL Connector (source & sink) dengan custom logic di DataStream API.

8. Gemini LLM: Insight Generation

Event anomali yang sudah terdeteksi (dari topic `traffic_anomaly_events`) dikonsumsi oleh service kecil yang memanggil Gemini untuk mengubah data numerik menjadi narasi yang actionable, contoh:

```
Input ke Gemini:
{
  "area": "Manhattan - Midtown",
  "condition": "heavy_rain",
  "trip_volume_change_pct": -42,
  "avg_fare_change_pct": +18,
  "window": "17:00-17:15"
}
```

```
Output Gemini (contoh):
"Volume trip di area Midtown turun 42% dan fare rata-rata naik 18% dalam 15 menit terakhir, bertepatan dengan hujan deras. Pola ini konsisten dengan kombinasi penurunan supply driver dan peningkatan demand jangka pendek. Rekomendasi: pantau area sekitar untuk potensi lonjakan permintaan serupa."
```

Hasil narasi ini yang dipresentasikan ke user/juri sebagai bagian dari kategori **Most Impactful AI App** — LLM di sini punya fungsi nyata (interpretasi & rekomendasi), bukan sekadar text-to-SQL generator.

9. Kesesuaian dengan Kategori Lomba

Kategori	Aspek yang Ditunjukkan
Most Flink-Driven App	SQL Source Connector (2 topic, beda format: Avro CDC vs plain JSON) + Interval Join (stateful) + Table/DataStream API interop + DataStream API (windowed anomaly detection) + SQL Sink Connector. Bukan cuma satu arah passthrough.
Most Impactful AI App	Gemini menerjemahkan hasil deteksi numerik menjadi insight & rekomendasi actionable untuk tim ops, bukan sekadar generate query.

10. Yang Masih Perlu Diputuskan

- Threshold/logic pasti untuk anomaly detection (fixed threshold vs z-score vs percentile).
- Sumber weather data final (Open-Meteo historical API vs dataset statis) dan granularity area (per zona TLC vs per koordinat).
- Arsitektur pemanggilan Gemini: batch per anomaly event, atau agregasi berkala (misal per 15 menit sekali panggil).

Dokumen ini adalah ringkasan working draft untuk keperluan hackathon internal, bukan spesifikasi final.